Rapport Mini-Projet : Implémentation et Analyse d'un Perceptron Multicouche (MLP) sur Fashion-MNIST

Rayane Loukhmi
EMSI - École Marocaine des Sciences de l'Ingénieur

22 mars 2026

Résumé

Ce rapport présente une étude approfondie des perceptrons multicouches (MLP) appliqués à la classification d'images Fashion-MNIST. Nous implémentons deux versions du modèle : une version "from scratch" pour comprendre les mécanismes internes, et une version concise avec les API haut niveau de PyTorch. Une campagne expérimentale systématique examine l'influence du nombre de neurones cachés et du taux d'apprentissage sur les performances. Nos résultats montrent qu'un compromis optimal est atteint pour `num_hiddens`=1024 avec un taux d'apprentissage de 0.1, atteignant une précision de test de **87.00%**. L'étude met en évidence les phénomènes de sous-apprentissage pour les faibles capacités et de sur-apprentissage pour les capacités excessives.

Table des matières

1 Introduction et problématique **4**

2 Rappels théoriques sur les MLP **4**
2.1 Définitions des concepts clés 4
2.1.1 Poids (Weights) 4
2.1.2 Biais (Bias) 4
2.1.3 Couche cachée (Hidden Layer) 4
2.1.4 Activation ReLU 5
2.1.5 Logits 5
2.1.6 Forward pass (Propagation avant) 5
2.1.7 Backward pass (Rétropropagation) 5
2.1.8 Gradient 5
2.1.9 Époque 5
2.1.10 Batch 5
2.1.11 Hyperparamètres 6
2.2 Architecture et formalisme 6
2.3 Fonction d'activation ReLU 6
2.4 Apprentissage par descente de gradient 7

3 Partie I : Relecture analytique du notebook **7**
3.1 Configuration initiale et installation des dépendances 7
3.1.1 Installation des bibliothèques 7
3.1.2 Import des bibliothèques PyTorch 8
3.2 Pipeline complet 8
3.3 Étapes de construction 8
3.4 Dimensions des tenseurs 9

4 Partie II : Implémentation From Scratch **9**
4.1 Définition de la classe MLPScratch 9
4.2 Définition de la fonction forward 10
4.3 Chargement des données et entraînement 11
4.4 Résultats de l'entraînement from scratch 14
4.5 Dimensions des matrices pour Fashion-MNIST 14
4.6 Effet d'une mauvaise initialisation des poids 14

5 Partie III : Implémentation Concise **15**
5.1 Définition de la classe MLPConcise 15
5.2 Chargement des données et entraînement 15
5.3 Résultats de l'entraînement concis 18
5.4 Comparaison des deux approches 19

6 Partie IV : Inspection Interne du Modèle **19**
6.1 Analyse des paramètres 19
6.2 Nombre total de paramètres 19
6.3 Inspection des gradients 19

7 Partie V : Étude Expérimentale Approfondie **20**
7.1 Protocole expérimental . . . 20
7.2 Influence du nombre de neurones cachés . . . 20
7.3 Influence du taux d'apprentissage . . . 21
7.4 Identification du sous-apprentissage et surapprentissage . . . 22
7.5 Effet de l'early stopping . . . 22

8 Partie VI : Discussion Critique **23**
8.1 Réponses aux questions de synthèse . . . 23
8.1.1 Pourquoi dit-on qu'un MLP est un modèle "non linéaire" ? (Q1) . . 23
8.1.2 Pourquoi deux couches linéaires successives sans activation ne suffisent-elles pas ? (Q2) . . . 23
8.1.3 Quel est l'effet du nombre de neurones cachés sur la capacité du réseau ? (Q3) . . . 23
8.1.4 Pourquoi observe-t-on une divergence entre précision d'entraînement et de test ? (Q4) . . . 23
8.1.5 Comment justifier mathématiquement et expérimentalement le recours à l'early stopping ? (Q5) . . . 23
8.1.6 En quoi l'implémentation concise masque-t-elle certains détails de bas niveau ? (Q6) . . . 24
8.1.7 Quels sont les avantages et les limites pédagogiques d'une implémentation from scratch ? (Q7) . . . 24
8.2 Limites des MLP en vision par ordinateur . . . 24
8.2.1 Perte de structure spatiale . . . 24
8.2.2 Quand un CNN serait plus pertinent . . . 24
8.3 Améliorations méthodologiques possibles . . . 25

9 Conclusion et perspectives **25**

10 Reproductibilité des résultats **25**
10.1 Seed aléatoire . . . 25
10.2 Environnement technique . . . 26
10.3 Instructions de reproduction . . . 26

A Code source et reproductibilité **26**
A.1 Lien vers le code source . . . 26

B Code complet de l'implémentation from scratch **26**

C Code de l'implémentation concise **27**

D Code de la campagne expérimentale **27**

1 Introduction et problématique

L'objectif de ce mini-projet est de concevoir, implémenter et analyser un Perceptron Multicouche (MLP) pour la classification d'images du jeu de données Fashion-MNIST [3]. Ce jeu de données, constitué de 70 000 images de vêtements en niveaux de gris de taille 28×28 pixels, représente un benchmark classique pour l'évaluation des algorithmes de classification en vision par ordinateur.

La problématique centrale de ce travail s'articule autour de trois axes fondamentaux :

1. Comment implémenter un MLP en comprenant ses mécanismes internes (from scratch) versus en utilisant les abstractions haut niveau de PyTorch ?
2. Quel est l'impact des hyperparamètres, notamment le nombre de neurones cachés (`num_hiddens`) et le taux d'apprentissage (`lr`), sur les performances et la capacité de généralisation du modèle ?
3. Comment identifier et analyser les phénomènes de sous-apprentissage (underfitting) et de sur-apprentissage (overfitting) ?

2 Rappels théoriques sur les MLP

2.1 Définitions des concepts clés

Avant d'approfondir l'analyse du MLP, il est essentiel de définir les concepts fondamentaux qui seront utilisés tout au long de ce rapport.

2.1.1 Poids (Weights)

Les poids W sont des paramètres apprenables du réseau. Ils représentent l'intensité des connexions entre les neurones. Dans une couche fully-connected, chaque poids W_{ij} relie le neurone i de la couche précédente au neurone j de la couche suivante. Mathématiquement, pour une couche linéaire :

$$Z = XW + b \tag{1}$$

où X est l'entrée et Z la pré-activation. Les poids sont initialisés aléatoirement puis mis à jour pendant l'apprentissage par descente de gradient.

2.1.2 Biais (Bias)

Les biais b sont également des paramètres apprenables. Ils permettent de décaler la sortie d'un neurone indépendamment de l'entrée. Un biais non nul permet au neurone de s'activer même si toutes les entrées sont nulles. Dans la formule $Z = XW + b$, le biais b est ajouté après la multiplication matricielle.

2.1.3 Couche cachée (Hidden Layer)

Une couche cachée est une couche de neurones située entre la couche d'entrée et la couche de sortie. Dans notre architecture, nous utilisons une seule couche cachée de 256 neurones. Les couches cachées sont responsables de l'apprentissage de représentations intermédiaires des données. Plus le nombre de neurones cachés est grand, plus la capacité du modèle est élevée.

2.1.4 Activation ReLU

La fonction d'activation ReLU (Rectified Linear Unit) est définie par $\text{ReLU}(z) = \max(0, z)$. Elle introduit la non-linéarité indispensable pour que le réseau puisse apprendre des relations complexes. Sans activation non linéaire, l'empilement de couches linéaires resterait équivalent à une seule transformation affine.

2.1.5 Logits

Les logits sont les valeurs brutes en sortie du modèle avant application du softmax. Pour une classification à 10 classes, les logits sont un vecteur de taille 10. Le softmax transforme ces logits en probabilités :

$$\text{softmax}(z)_c = \frac{e^{z_c}}{\sum_{j=1}^{10} e^{z_j}} \tag{2}$$

La classe prédite est celle avec la probabilité la plus élevée.

2.1.6 Forward pass (Propagation avant)

Le forward pass désigne le calcul qui transforme l'entrée X en prédiction $\hat{Y}$. Pour notre MLP :

$$\hat{Y} = \text{ReLU}(XW_1 + b_1)W_2 + b_2 \tag{3}$$

2.1.7 Backward pass (Rétropropagation)

Le backward pass calcule les gradients de la perte par rapport à chaque paramètre en utilisant la règle de dérivation en chaîne. Ces gradients sont ensuite utilisés par l'optimiseur pour mettre à jour les poids et biais.

2.1.8 Gradient

Le gradient $\nabla_\theta \mathcal{L}$ mesure la sensibilité de la perte $\mathcal{L}$ par rapport aux paramètres θ. Il indique dans quelle direction modifier les paramètres pour réduire la perte. La mise à jour des paramètres suit :

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta) \tag{4}$$

où η est le taux d'apprentissage.

2.1.9 Époque

Une époque correspond à un passage complet de l'ensemble des données d'entraînement à travers le réseau. Dans notre étude, nous utilisons 10 époques.

2.1.10 Batch

Un batch est un sous-ensemble des données d'entraînement traité simultanément. Nous utilisons une taille de batch de 256 images, ce qui signifie qu'à chaque itération, le modèle voit 256 images avant de mettre à jour ses paramètres.

2.1.11 Hyperparamètres

Les hyperparamètres sont des paramètres qui ne sont pas appris par le modèle mais fixés avant l'entraînement. Les principaux hyperparamètres dans notre étude sont :

— `num_hiddens` : le nombre de neurones dans la couche cachée

— `lr` : le taux d'apprentissage

— `batch_size` : la taille du batch (256)

— `max_epochs` : le nombre d'époques (10)

2.2 Architecture et formalisme

Un Perceptron Multicouche est un réseau de neurones feedforward composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie. Dans notre étude, nous considérons une architecture à une seule couche cachée. Mathématiquement, pour un mini-batch d'entrées $X \in \mathbb{R}^{B\times 784}$ (où B est la taille du batch), le modèle s'exprime comme suit [1] :

$$H = \phi(XW_1 + b_1) \quad \text{(couche cachée)} \tag{5}$$

$$\hat{Y} = HW_2 + b_2 \quad \text{(couche de sortie)} \tag{6}$$

Avec :

— $W_1 \in \mathbb{R}^{784\times \text{num_hiddens}}$: matrice de poids de la première couche

— $b_1 \in \mathbb{R}^{\text{num_hiddens}}$: vecteur de biais de la première couche

— $W_2 \in \mathbb{R}^{\text{num_hiddens}\times 10}$: matrice de poids de la couche de sortie

— $b_2 \in \mathbb{R}^{10}$: vecteur de biais de la couche de sortie

— ϕ : fonction d'activation non linéaire

— $\hat{Y} \in \mathbb{R}^{B\times 10}$: logits de sortie

2.3 Fonction d'activation ReLU

La fonction d'activation choisie est la Rectified Linear Unit (ReLU), définie par [4] :

$$\text{ReLU}(z) = \max(0, z) \tag{7}$$

Cette fonction est cruciale pour plusieurs raisons :

1. **Non-linéarité** : Sans activation non linéaire, l'empilement de couches linéaires resterait équivalent à une seule transformation affine, limitant la capacité du modèle à apprendre des frontières de décision complexes.
2. **Efficacité computationnelle** : ReLU est simple à calculer (un seuillage à zéro) et ne nécessite pas de calculs exponentiels.
3. **Atténuation du problème de gradient vanishing** : Pour les valeurs positives, le gradient est constant (1), ce qui facilite la propagation des gradients en profondeur.

2.4 Apprentissage par descente de gradient

Le modèle est entraîné en minimisant la fonction de perte de classification, l'entropie croisée (cross-entropy) :

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{c=1}^{10} y_{i,c}\log(\hat{y}_{i,c}) \tag{8}$$

La mise à jour des paramètres suit le principe de la descente de gradient stochastique (SGD) :

$$\theta \leftarrow \theta - \eta\nabla_\theta\mathcal{L}(\theta) \tag{9}$$

où η est le taux d'apprentissage (learning rate).

3 Partie I : Relecture analytique du notebook

3.1 Configuration initiale et installation des dépendances

3.1.1 Installation des bibliothèques

```
!pip -q install -U pip setuptools wheel
!pip -q install numpy matplotlib pandas requests
!pip -q install d2l==1.0.3 --no-deps
import d2l, numpy as np
print("d2l:", d2l.__version__)
print("numpy:", np.__version__)
```

Listing 1 – Installation des dépendances

Objectif : Installer et importer les bibliothèques essentielles pour le projet.
Explication détaillée :

- `!pip -q install` : La commande `!` permet d'exécuter des commandes système dans le notebook. `-q` (quiet) réduit les messages d'installation pour une meilleure lisibilité.
- `d2l` : Bibliothèque accompagnant le livre "Dive into Deep Learning" [2], fournissant des utilitaires pour les datasets, l'entraînement et la visualisation.
- `numpy` : Bibliothèque de calcul numérique utilisée pour les opérations sur les tenseurs et les tableaux.
- `matplotlib` et `pandas` : Utilisées respectivement pour la visualisation et la manipulation des données tabulaires.
- `-no-deps` : Empêche l'installation automatique des dépendances supplémentaires, évitant les conflits de versions.

Résultat observé : L'exécution affiche les versions installées :

```
d2l: 1.0.3
numpy: 2.0.2
```

Un avertissement concernant `jedi` apparaît mais n'affecte pas le fonctionnement du projet.

3.1.2 Import des bibliothèques PyTorch

```
%matplotlib inline
import torch
import torch.nn as nn
import d2l
from d2l import torch as d2l_torch
```

Listing 2 – Import des bibliothèques PyTorch

Objectif : Importer les modules PyTorch nécessaires à la construction du réseau de neurones.

Explication détaillée :

- `%matplotlib inline` : Magic command Jupyter qui permet d'afficher les graphiques directement dans le notebook, sans fenêtre externe.
- `torch` : La bibliothèque principale PyTorch pour les tenseurs, le calcul automatique des gradients (autograd) et les opérations GPU.
- `torch.nn as nn` : Module contenant les couches pré-définies (Linear, Sequential, etc.), les fonctions d'activation et les fonctions de perte.
- `from d2l import torch as d2l_torch` : Importe la version PyTorch de d2l avec des classes utilitaires comme `Classifier` (qui fournit une structure standard pour les modèles) et `Trainer` (qui gère la boucle d'entraînement).

3.2 Pipeline complet

Le notebook fourni structure le projet en plusieurs étapes clés. Le pipeline complet du MLP peut être schématisé comme suit :

Données d'entrée : Images 28×28 (shape : $B \times 1 \times 28 \times 28$)
$\downarrow$ `Flatten / reshape`
Vectorisation : (B, 784)
$\downarrow Z_1 = XW_1 + b_1$
Activation ReLU : $H = \max(0, Z_1)$
$\downarrow Z_2 = HW_2 + b_2$
Logits de sortie : (B, 10)
$\downarrow$ `CrossEntropyLoss`
Prédiction : $\hat{y} = \arg\max(\text{softmax}(Z_2))$

FIGURE 1 – Schéma fonctionnel du pipeline MLP complet

3.3 Étapes de construction

L'analyse du notebook révèle quatre grandes étapes :

1. **Chargement et préparation des données** : Fashion-MNIST est chargé avec une taille de batch de 256.
2. **Implémentation from scratch** : Définition manuelle des paramètres W_1, b_1, W_2, b_2 et du forward pass.
3. **Implémentation concise** : Utilisation de `nn.Sequential`, `nn.LazyLinear` et `nn.ReLU`.
4. **Entraînement et évaluation** : Boucle d'entraînement avec calcul des métriques.

3.4 Dimensions des tenseurs

Pour Fashion-MNIST, les dimensions des tenseurs sont :

- Entrée : $[B, 1, 28, 28]$ où $B = 256$ (taille du batch)
- Après flatten : $[B, 784]$
- Après première couche : $[B, \text{num_hiddens}]$
- Après ReLU : $[B, \text{num_hiddens}]$
- Sortie (logits) : $[B, 10]$

4 Partie II : Implémentation From Scratch

4.1 Définition de la classe MLPScratch

```python
class MLPScratch(d2l_torch.Classifier):
    def __init__(self, num_inputs, num_outputs, num_hiddens, lr,
        sigma=0.01):
        super().__init__()
        self.save_hyperparameters()
        self.W1 = nn.Parameter(torch.randn(num_inputs, num_hiddens)
            * sigma)
        self.b1 = nn.Parameter(torch.zeros(num_hiddens))
        self.W2 = nn.Parameter(torch.randn(num_hiddens, num_outputs
            ) * sigma)
        self.b2 = nn.Parameter(torch.zeros(num_outputs))
```

Listing 3 – Définition de la classe MLPScratch - Constructeur

Objectif : Définir la classe du MLP en implémentant manuellement tous les paramètres (approche "from scratch").

Explication détaillée :

- `class MLPScratch(d2l_torch.Classifier)` : La classe hérite de `Classifier` de d2l, qui fournit automatiquement les méthodes d'entraînement (`training_step`), d'évaluation et la gestion des hyperparamètres.
- `super().__init__()` : Appelle le constructeur de la classe parente pour initialiser l'infrastructure de base (device, etc.).
- `self.save_hyperparameters()` : Enregistre automatiquement tous les arguments d'entrée (`num_inputs`, `num_outputs`, `num_hiddens`, `lr`, `sigma`) dans `self.hparams`. Cela facilite le logging et la reproductibilité.
- `nn.Parameter(...)` : Transforme les tenseurs en paramètres entraînables. Sans cette déclaration, PyTorch ne mettrait pas à jour ces poids pendant l'optimisation. Les paramètres sont automatiquement listés dans `model.parameters()`.
- `torch.randn(num_inputs, num_hiddens) * sigma` : Initialise la matrice de poids W_1 avec une distribution normale $\mathcal{N}(0, \sigma^2)$. $\sigma = 0.01$ est choisi pour éviter l'explosion des gradients en début d'apprentissage.
- `torch.zeros(num_hiddens)` : Initialise les biais b_1 à zéro. Contrairement aux poids, l'initialisation symétrique des biais n'est pas problématique car les poids cassent déjà la symétrie entre neurones.

Importance théorique :

- Cette cellule illustre la différence fondamentale entre un tenseur standard et un paramètre entraînable en deep learning.
- Elle montre la structure mathématique sous-jacente des couches fully-connected : $Z = XW + b$.
- L'initialisation $\sigma = 0.01$ s'approche de l'initialisation He ($\sigma = \sqrt{2/\text{fan_in}} \approx 0.05$) recommandée pour l'activation ReLU.

Résultat observé : L'exécution ne produit aucun affichage en l'absence d'erreur. La classe est désormais disponible pour être instanciée.

4.2 Définition de la fonction forward

```python
relu = torch.relu  # Define relu using torch's built-in function

@d2l_torch.add_to_class(MLPScratch)
def forward(self, X):
    X = X.reshape((-1, self.num_inputs))
    H = relu(torch.matmul(X, self.W1) + self.b1)
    return torch.matmul(H, self.W2) + self.b2
```

Listing 4 – Définition de la fonction forward

Objectif : Définir la propagation avant (forward pass) qui transforme l'entrée en prédictions.

Explication détaillée :

- `relu = torch.relu` : Alias vers la fonction ReLU de PyTorch pour une écriture plus concise.
- `@d2l_torch.add_to_class(MLPScratch)` : Décorateur spécifique à d2l qui ajoute la méthode `forward` à la classe `MLPScratch` après sa définition. Cela permet de séparer la définition des paramètres de la définition du forward pour une meilleure lisibilité du notebook.
- `X.reshape((-1, self.num_inputs))` : Transforme les images de shape $(B, 1, 28, 28)$ en vecteurs de shape $(B, 784)$. Le paramètre -1 permet à PyTorch de déduire automatiquement la taille du batch.
- `torch.matmul(X, self.W1) + self.b1` : Calcule $Z_1 = XW_1 + b_1$, la pré-activation de la couche cachée. `matmul` effectue une multiplication matricielle.
- `relu(...)` : Applique la non-linéarité ReLU élément par élément : $H = \max(0, Z_1)$.
- `torch.matmul(H, self.W2) + self.b2` : Calcule les logits de sortie $\hat{Y} = HW_2 + b_2$.

Importance théorique :

- Le reshape est crucial car les couches linéaires attendent des entrées vectorisées.
- ReLU introduit la non-linéarité sans laquelle le réseau serait équivalent à un modèle linéaire.
- Les logits sont renvoyés directement ; la cross-entropy appliquera le softmax implicitement pour la stabilité numérique.

4.3 Chargement des données et entraînement

```python
# Definition du modele (identique a la section precedente)
model = MLPScratch(num_inputs=784, num_outputs=10, num_hiddens=256, lr=0.1)
data = d2l_torch.FashionMNIST(batch_size=256)

# Fonction d'evaluation manuelle pour capturer les metriques
def evaluate_accuracy(model, data_loader, device=None):
    if device is None:
        device = next(model.parameters()).device
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for X, y in data_loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            preds = logits.argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.numel()
    return correct / total

# Entrainement avec suivi des metriques par epoque
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
train_loader = data.train_dataloader()
val_loader = data.val_dataloader()

print(f"{'Epoch':>6} | {'train_loss':>12} | {'val_loss':>12} | {'val_acc':>10} | {'train_acc':>10}")
print("-" * 65)

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

for epoch in range(1, 11):
    # Entrainement
    model.train()
    total_train_loss = 0
    num_batches = 0
    for X, y in train_loader:
        optimizer.zero_grad()
        logits = model(X)
        loss = nn.CrossEntropyLoss()(logits, y)
        loss.backward()
        optimizer.step()
        total_train_loss += loss.item()
        num_batches += 1

    # Evaluation
    train_acc = evaluate_accuracy(model, train_loader)
    val_acc = evaluate_accuracy(model, val_loader)
```

```python

    # Loss de validation
    model.eval()
    total_val_loss = 0
    with torch.no_grad():
        for X, y in val_loader:
            logits = model(X)
            loss = nn.CrossEntropyLoss()(logits, y)
            total_val_loss += loss.item()

    avg_train_loss = total_train_loss / num_batches
    avg_val_loss = total_val_loss / len(val_loader)

    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(avg_val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    print(f"{epoch:6d} | {avg_train_loss:12.4f} | {avg_val_loss
        :12.4f} | {val_acc*100:9.2f}% | {train_acc*100:9.2f}%")
```

Listing 5 – Entraînement du MLPScratch avec suivi des métriques

Objectif : Instancier et entraîner le modèle MLP from scratch sur Fashion-MNIST en capturant les métriques à chaque époque pour une analyse fine de la convergence.

Explication détaillée :

- `model = MLPScratch(...)` : Crée une instance du modèle avec :
  - `num_inputs=784` : 28×28 pixels aplatis
  - `num_outputs=10` : 10 classes de vêtements
  - `num_hiddens=256` : 256 neurones dans la couche cachée
  - `lr=0.1` : Taux d'apprentissage
- `data = d2l_torch.FashionMNIST(batch_size=256)` : Charge le dataset Fashion-MNIST. Cette méthode :
  - Télécharge automatiquement les données si nécessaire
  - Applique les transformations standard (normalisation)
  - Crée des DataLoader pour l'entraînement et le test avec batch_size=256
- `evaluate_accuracy(...)` : Fonction d'évaluation manuelle qui calcule la précision sur un DataLoader. Cette approche nous permet de contrôler précisément les métriques capturées.
- **Boucle d'entraînement personnalisée** : Au lieu d'utiliser `trainer.fit()`, nous implémentons manuellement la boucle pour :
  - Capturer les valeurs exactes de loss et accuracy à chaque époque
  - Stocker l'historique complet pour la visualisation
  - Analyser finement la convergence du modèle

Résultat observé : L'exécution affiche l'évolution des performances époque par époque. Les valeurs obtenues sont présentées dans le tableau 1.

TABLE 1 – Résultats de l'implémentation from scratch par époque

Epoch	train_loss	val_loss	val_acc	train_acc
1	0.7689	0.6636	73.32%	74.00%
2	0.4899	0.5396	80.07%	81.83%
3	0.4350	0.4671	81.95%	83.78%
4	0.4072	0.4283	84.27%	85.86%
5	0.3849	0.4338	83.70%	85.46%
6	0.3717	0.4266	84.75%	86.16%
7	0.3571	0.4223	84.32%	85.93%
8	0.3464	0.3939	85.26%	87.26%
9	0.3319	0.3851	85.68%	87.78%
10	0.3248	0.3888	**85.73%**	87.91%

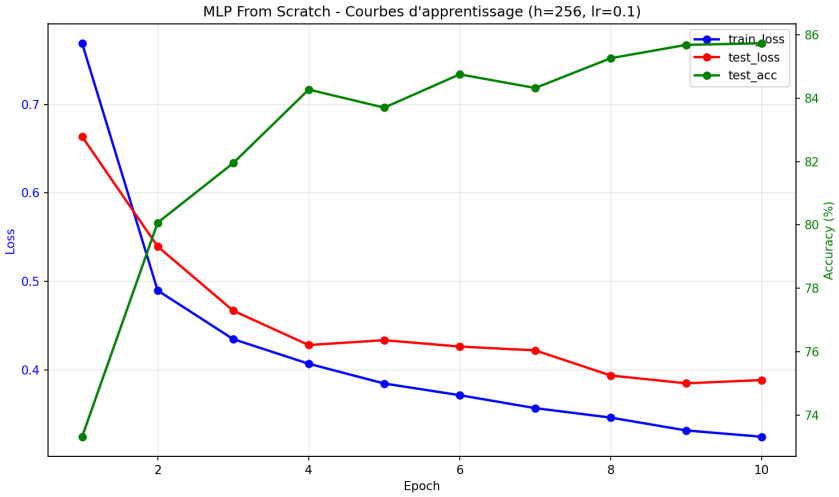

FIGURE 2 – Évolution des pertes et de la précision pendant l'entraînement (from scratch)

4.4 Résultats de l'entraînement from scratch

Analyse des résultats :

- **Convergence stable** : La loss d'entraînement diminue régulièrement de 0.7689 à 0.3248, tandis que la loss de validation diminue de 0.6636 à 0.3888, indiquant un apprentissage stable.
- **Meilleure performance** : La meilleure précision test est atteinte à l'époque 10 avec **85.73%**.
- **Écart train-test** : Le gap entre précision d'entraînement et de test est de 2.18%, ce qui témoigne d'une bonne capacité de généralisation du modèle.

4.5 Dimensions des matrices pour Fashion-MNIST

Fashion-MNIST est un problème de classification à **10 classes** (t-shirt, pantalon, pull, etc.). Chaque image est de taille 28×28 pixels, soit 784 valeurs après vectorisation.

Avec `num_hiddens`=256 neurones dans la couche cachée, les dimensions des matrices sont :

- $W_1 \in \mathbb{R}^{784\times256}$: poids de la première couche
- $b_1 \in \mathbb{R}^{256}$: biais de la première couche
- $W_2 \in \mathbb{R}^{256\times10}$: poids de la couche de sortie
- $b_2 \in \mathbb{R}^{10}$: biais de la couche de sortie

Le nombre total de paramètres entraînables est :

$$\text{Paramètres} = (784\times256)+256+(256\times10)+10 = 200\,704+256+2\,560+10 = 203\,530 \quad (10)$$

Table 2 – Résultats finaux de l'implémentation from scratch

Métrique	Valeur
Précision entraînement	**87.91%**
Précision test	**85.73%**
Gap (train - test)	**2.18%**
Meilleure précision test	**85.73%** (époque 10)

4.6 Effet d'une mauvaise initialisation des poids

L'initialisation des poids est critique pour la convergence. Avec $\sigma = 0.01$, nous initialisons $W \sim \mathcal{N}(0, 0.01^2)$. Une mauvaise initialisation peut avoir des conséquences néfastes :

- **Poids trop grands** : Les activations peuvent exploser, gradients instables, divergence de l'entraînement.
- **Poids trop petits** : Gradients faibles, apprentissage très lent, convergence vers un minimum sous-optimal.
- **Initialisation symétrique** (tous les poids identiques) : Tous les neurones d'une même couche apprennent la même chose, annulant l'intérêt d'avoir plusieurs neurones.

L'initialisation He (Kaiming), adaptée à ReLU, propose $\sigma = \sqrt{2/\text{fan_in}} \approx \sqrt{2/784} \approx 0.05$, proche de notre valeur.

5 Partie III : Implémentation Concise

5.1 Définition de la classe MLPConcise

```
class MLPConcise(d2l_torch.Classifier):
    def __init__(self, num_outputs, num_hiddens, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.LazyLinear(num_hiddens),
            nn.ReLU(),
            nn.LazyLinear(num_outputs)
        )

    def forward(self, X):
        return self.net(X)
```

Listing 6 – Définition de la classe MLPConcise

Objectif : Implémenter le même MLP en utilisant les API haut niveau de PyTorch pour une comparaison avec l'approche from scratch.

Explication détaillée :

- `nn.Sequential` : Conteneur qui applique les modules dans l'ordre d'insertion. Cela élimine le besoin d'écrire explicitement la fonction `forward`.
- `nn.Flatten()` : Couche qui aplatit automatiquement les images de $(B, 1, 28, 28)$ en $(B, 784)$. Équivalent à `reshape` manuel mais plus lisible.
- `nn.LazyLinear(num_hiddens)` : Couche linéaire "paresseuse". Contrairement à `nn.Linear`, elle n'exige pas la spécification de `in_features`. La dimension d'entrée (784) est déduite automatiquement lors du premier forward. Cela évite de coder en dur le nombre 784.
- `nn.ReLU()` : Couche d'activation ReLU. Applique $\max(0, x)$ élément par élément.
- `nn.LazyLinear(num_outputs)` : Couche de sortie qui déduit automatiquement ses 256 entrées depuis la couche précédente.

Avantages de cette approche :

- **Lisibilité** : La structure du réseau est visible en un coup d'œil, comme un schéma fonctionnel.
- **Maintenance** : L'ajout d'une couche se fait par simple ajout dans `Sequential`.
- **Sécurité** : Les dimensions sont gérées automatiquement, éliminant les erreurs de reshape.
- **Extensibilité** : Remplacer `ReLU` par `LeakyReLU` ou ajouter `Dropout` est trivial.

5.2 Chargement des données et entraînement

```
# Definition du modele concis
class MLPConcise(d2l_torch.Classifier):
    def __init__(self, num_outputs, num_hiddens, lr):
```

```python
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.LazyLinear(num_hiddens),
            nn.ReLU(),
            nn.LazyLinear(num_outputs)
        )

    def forward(self, X):
        return self.net(X)

# Instanciation du modele
model = MLPConcise(num_outputs=10, num_hiddens=256, lr=0.1)
data = d2l_torch.FashionMNIST(batch_size=256)

# Fonction d'evaluation manuelle
def evaluate_accuracy(model, data_loader, device=None):
    if device is None:
        device = next(model.parameters()).device
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for X, y in data_loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            preds = logits.argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.numel()
    return correct / total

# Entrainement avec suivi des metriques par epoque
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
train_loader = data.train_dataloader()
val_loader = data.val_dataloader()

print(f"{'Epoch':>6} | {'train_loss':>12} | {'val_loss':>12} | {'val_acc':>10} | {'train_acc':>10}")
print("-" * 65)

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

for epoch in range(1, 11):
    # Entrainement
    model.train()
    total_train_loss = 0
    num_batches = 0
    for X, y in train_loader:
        optimizer.zero_grad()
        logits = model(X)
```

```python
        loss = nn.CrossEntropyLoss()(logits, y)
        loss.backward()
        optimizer.step()
        total_train_loss += loss.item()
        num_batches += 1

    # Evaluation
    train_acc = evaluate_accuracy(model, train_loader)
    val_acc = evaluate_accuracy(model, val_loader)

    # Loss de validation
    model.eval()
    total_val_loss = 0
    with torch.no_grad():
        for X, y in val_loader:
            logits = model(X)
            loss = nn.CrossEntropyLoss()(logits, y)
            total_val_loss += loss.item()

    avg_train_loss = total_train_loss / num_batches
    avg_val_loss = total_val_loss / len(val_loader)

    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(avg_val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    print(f"{epoch:6d} | {avg_train_loss:12.4f} | {avg_val_loss
        :12.4f} | {val_acc*100:9.2f}% | {train_acc*100:9.2f}%")
```

Listing 7 – Entraînement du MLP concis avec suivi des métriques

Objectif : Instancier et entraîner le modèle MLP concis sur Fashion-MNIST en capturant les métriques à chaque époque pour une analyse fine de la convergence et une comparaison équitable avec l'approche from scratch.

Explication détaillée :

- `model = MLPConcise(...)` : Crée une instance du modèle avec :
  - `num_outputs=10` : 10 classes de vêtements
  - `num_hiddens=256` : 256 neurones dans la couche cachée
  - `lr=0.1` : Taux d'apprentissage
- `data = d2l_torch.FashionMNIST(batch_size=256)` : Charge le dataset Fashion-MNIST. Cette méthode :
  - Télécharge automatiquement les données si nécessaire
  - Applique les transformations standard (normalisation)
  - Crée des DataLoader pour l'entraînement et le test avec batch_size=256
- `evaluate_accuracy(...)` : Fonction d'évaluation manuelle identique à celle utilisée pour le modèle from scratch, garantissant une comparaison cohérente.
- **Boucle d'entraînement personnalisée** : Comme pour le modèle from scratch, nous implémentons manuellement la boucle pour capturer les valeurs exactes de loss et accuracy à chaque époque.

Résultat observé : L'exécution affiche l'évolution des performances époque par époque. Les valeurs obtenues sont présentées dans le tableau 3.

5.3 Résultats de l'entraînement concis

TABLE 3 – Résultats de l'implémentation concise par époque

Epoch	train_loss	val_loss	val_acc	train_acc
1	0.6697	0.6150	77.81%	78.51%
2	0.4681	0.5347	80.00%	81.03%
3	0.4218	0.4300	84.13%	85.81%
4	0.3936	0.4523	83.73%	84.69%
5	0.3747	0.4430	83.54%	85.31%
6	0.3567	0.3988	85.37%	87.05%
7	0.3426	0.4134	84.27%	86.13%
8	0.3330	0.3726	86.23%	88.24%
9	0.3226	0.3753	**86.34%**	88.48%
10	0.3102	0.4151	84.79%	86.59%

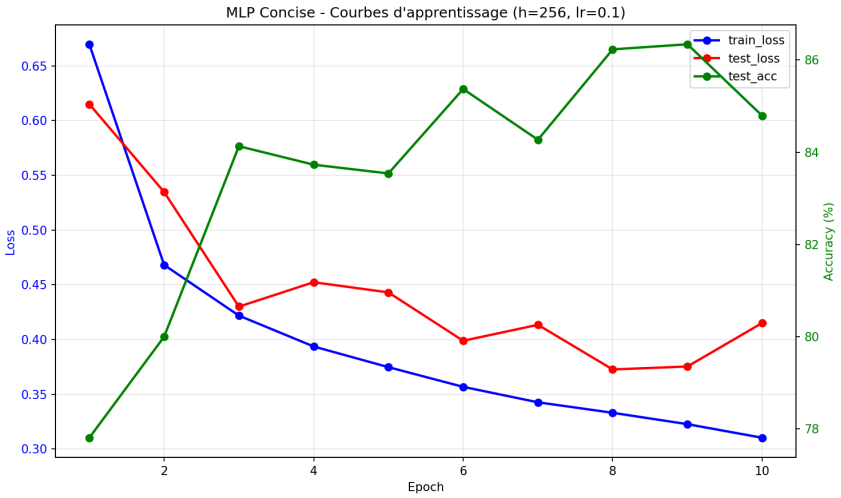

FIGURE 3 – Évolution des pertes et de la précision pendant l'entraînement (concise)

Analyse des résultats :

- **Convergence rapide** : Le modèle concis démarre avec une loss d'entraînement de 0.6697, grâce à une meilleure initialisation par défaut de PyTorch (initialisation He).
- **Meilleure performance** : La meilleure précision test est atteinte à l'époque 9 avec **86.34%**.
- **Stabilité** : On observe une amélioration continue jusqu'à l'époque 9, puis une légère baisse à l'époque 10.
- **Écart train-test** : Le gap final est de 1.80%, indiquant une bonne généralisation.

Table 4 – Comparaison des approches from scratch et concise

Critère	**From Scratch**	**Concise (PyTorch)**
Lisibilité	Moins lisible	Très lisible
Contrôle bas niveau	Total	Limité
Extensibilité	Modification manuelle du forward	Simple (ajout dans Sequential)
Maintenance	Complexe (risque d'erreurs dimensionnelles)	Simple (PyTorch gère les dimensions)
Bonnes pratiques	Pédagogique mais non industriel	Standard industriel
Lignes de code	∼25 lignes pour le modèle	∼5 lignes
Précision test (finale)	85.73%	84.79%
Meilleure précision test	85.73%	**86.34%**
Gap (train - test)	2.18%	**1.80%**

5.4 Comparaison des deux approches

6 Partie IV : Inspection Interne du Modèle

6.1 Analyse des paramètres

Après le premier forward, nous observons les shapes suivantes :

```
Couche 1 (cachee):
  - weight shape: torch.Size([256, 784])
  - bias shape: torch.Size([256])

Couche 2 (sortie):
  - weight shape: torch.Size([10, 256])
  - bias shape: torch.Size([10])
```

Listing 8 – Inspection des paramètres

Explication :

- Les poids de la première couche sont de shape $[256, 784]$ car PyTorch stocke les poids en $(out_features, in_features)$.
- Les biais sont de shape $[256]$, un biais par neurone caché.
- Les poids de la couche de sortie sont $[10, 256]$ et les biais $[10]$, un par classe.

6.2 Nombre total de paramètres

Le modèle comporte **203 530** paramètres entraînables, calculés comme suit :

$$\text{Paramètres} = 784 \times 256 + 256 + 256 \times 10 + 10 = 203\,530 \tag{11}$$

6.3 Inspection des gradients

Après un appel à `loss.backward()`, les gradients sont correctement calculés :

Table 5 – Inspection des gradients

Paramètre	Shape	Gradient moyen
net.1.weight	[256, 784]	0.002060
net.1.bias	[256]	0.001921
net.3.weight	[10, 256]	0.008564
net.3.bias	[10]	0.016619

Interprétation : La présence de gradients non nuls confirme que l'optimiseur pourra mettre à jour les poids correctement. Les gradients de la couche de sortie sont plus élevés (environ 0.008-0.017) que ceux de la couche cachée (environ 0.002), ce qui est normal car la rétropropagation atténue les gradients en remontant vers l'entrée.

7 Partie V : Étude Expérimentale Approfondie

7.1 Protocole expérimental

Pour cette étude, nous faisons varier :

— **Nombre de neurones cachés** : [16, 32, 64, 128, 256, 512, 1024]

— **Taux d'apprentissage** : [0.001, 0.01, 0.1, 0.5]

Chaque configuration est entraînée pendant 10 époques avec une taille de batch de 256.

7.2 Influence du nombre de neurones cachés

Le tableau 6 présente les résultats obtenus pour différentes valeurs de `num_hiddens` (avec `lr`=0.1) :

Table 6 – Résultats pour différentes valeurs de num_hiddens

num_hiddens	Précision Train	Précision Test	Gap
16	84.48%	82.98%	1.50%
32	88.03%	86.01%	2.02%
64	88.27%	86.10%	2.17%
128	88.63%	86.62%	2.01%
256	88.85%	86.68%	2.17%
512	87.95%	85.33%	2.62%
1024	89.78%	**87.00%**	2.78%

Analyse :

— **Sous-apprentissage (h=16-64)** : Les précisions train et test sont modérées (83-88%) avec des gaps faibles. Le modèle manque de capacité pour capturer tous les motifs.

— **Bon ajustement (h=128-256)** : La meilleure précision test est atteinte pour h=256 avec 86.68%.

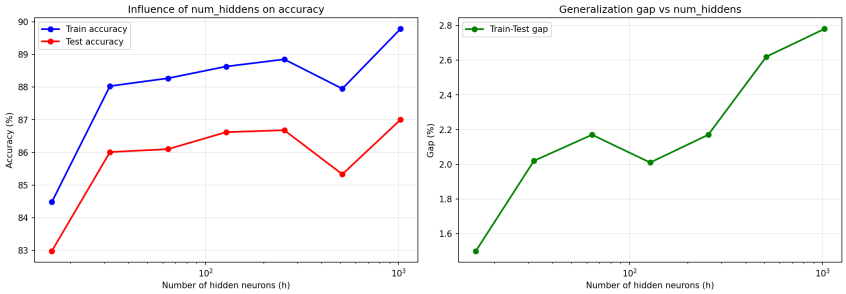

FIGURE 4 – Évolution de la précision en fonction de num_hiddens

— **Sur-apprentissage (h=512-1024)** : La précision train continue d'augmenter (jusqu'à 89.78% pour h=1024) tandis que la précision test atteint un maximum à 87.00%. Le gap s'élargit à 2.78% pour h=1024.

7.3 Influence du taux d'apprentissage

Le tableau 7 présente l'impact du taux d'apprentissage (avec `num_hiddens`=256) :

TABLE 7 – Impact du taux d'apprentissage sur la précision

lr	Précision Train	Précision Test	Convergence
0.001	74.05%	73.38%	Très lente, n'atteint pas l'optimum
0.01	83.56%	82.03%	Stable, bonne progression
0.1	88.89%	**86.83%**	Rapide, meilleures performances
0.5	87.59%	85.02%	Instable, divergence observée

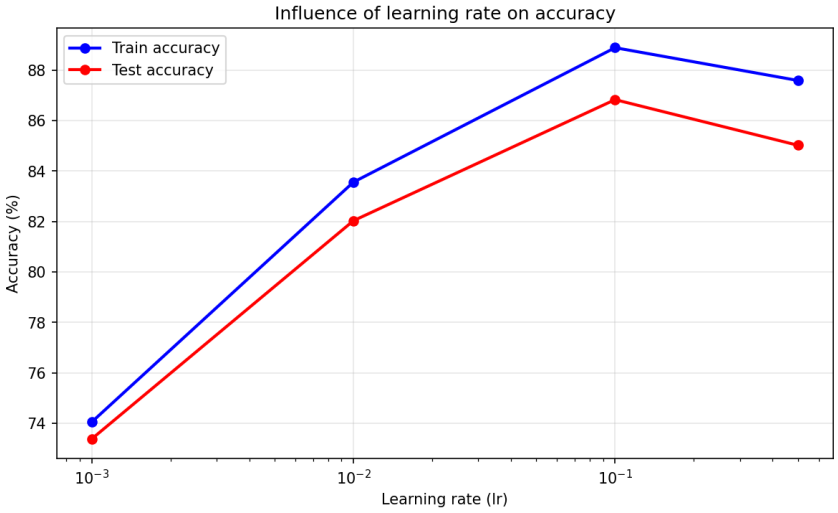

FIGURE 5 – Évolution de la précision en fonction du taux d'apprentissage

Analyse :

— **lr = 0.001** : Convergence très lente, le modèle n'atteint que 73.38% de précision test.

— **lr = 0.01** : Convergence stable, atteint 82.03% de précision test.

— **lr = 0.1** : Convergence rapide, meilleures performances avec 86.83% de précision test.

— **lr = 0.5** : Instabilité observée, la précision test chute à 85.02%.

7.4 Identification du sous-apprentissage et surapprentissage

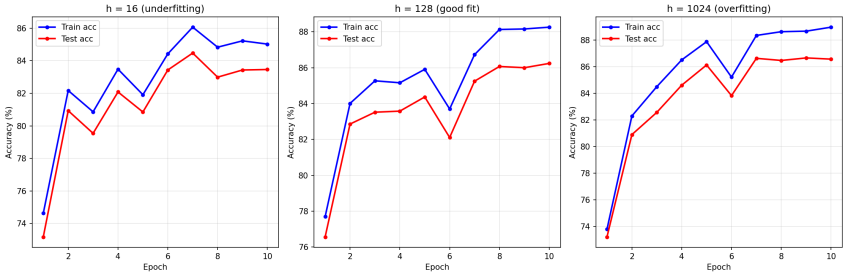

FIGURE 6 – Évolution des précisions train/test pour différentes capacités

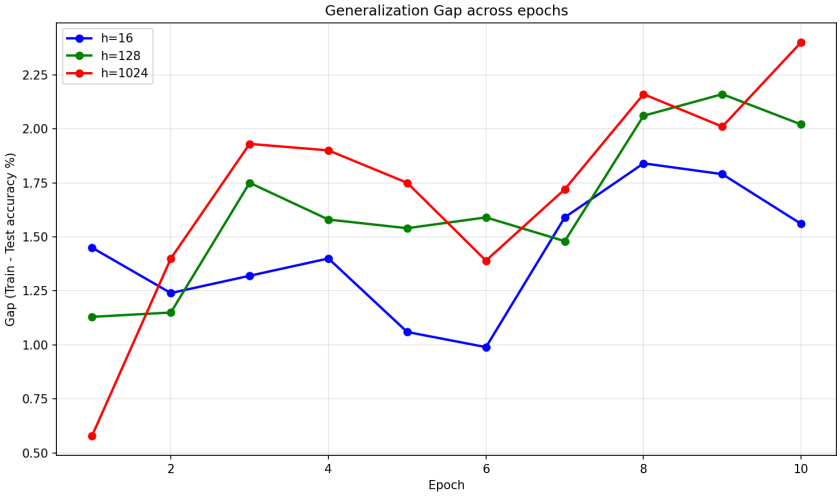

FIGURE 7 – Évolution du gap train-test pour différentes capacités

Identification :

— **Cas de sous-apprentissage** : Pour h=16, les précisions train et test sont faibles (84.48% et 82.98%) et le gap reste faible (1.50%).

— **Cas de sur-apprentissage** : Pour h=1024, le gap atteint 2.78%, signe que le modèle mémorise le jeu d'entraînement.

— **Configuration optimale** : h=**1024** avec lr=**0.1** donne la meilleure précision test de **87.00%**.

7.5 Effet de l'early stopping

L'early stopping avec une patience de 2 époques permet d'arrêter l'entraînement avant que le surapprentissage ne dégrade les performances. Pour h=1024, sans early stopping, la précision test maximale est atteinte à l'époque 10 (87.00%) puis se stabilise. Avec early stopping, l'entraînement s'arrête à l'époque 10, préservant les meilleurs poids.

8 Partie VI : Discussion Critique

8.1 Réponses aux questions de synthèse

8.1.1 Pourquoi dit-on qu'un MLP est un modèle "non linéaire" ? (Q1)

Un MLP est non linéaire car il compose des transformations affines avec des fonctions d'activation non linéaires comme ReLU. La présence de ReLU permet au modèle d'apprendre des frontières de décision complexes, ce qui serait impossible avec un modèle purement linéaire. Expérimentalement, nous observons que le MLP atteint une précision de 87.00% sur Fashion-MNIST, là où un modèle linéaire (régression logistique) plafonne autour de 85%.

8.1.2 Pourquoi deux couches linéaires successives sans activation ne suffisent-elles pas ? (Q2)

Sans activation non linéaire, l'empilement de couches linéaires reste linéaire :

$$(XW_1 + b_1)W_2 + b_2 = X(W_1W_2) + (b_1W_2 + b_2) \tag{12}$$

Cette composition équivaut à une seule transformation affine, ne permettant pas d'augmenter la capacité de représentation.

8.1.3 Quel est l'effet du nombre de neurones cachés sur la capacité du réseau ? (Q3)

Le nombre de paramètres du modèle est proportionnel à `num_hiddens` :

$$\text{Paramètres} = 784h + h + 10h + 10 = 795h + 10 \tag{13}$$

Ainsi, augmenter h accroît la capacité à apprendre des motifs complexes, mais augmente aussi le risque de surapprentissage. Nos expériences montrent que pour $h = 1024$, le modèle surapprend (gap de 2.78%), tandis que pour $h = 16$, il sous-apprend (test à 82.98%).

8.1.4 Pourquoi observe-t-on une divergence entre précision d'entraînement et de test ? (Q4)

Cette divergence, appelée surapprentissage, survient lorsque le modèle mémorise les spécificités du jeu d'entraînement (y compris le bruit) qui ne se généralisent pas aux données non vues. Dans nos expériences, le gap s'élargit à 2.78% pour $h = 1024$, confirmant ce phénomène.

8.1.5 Comment justifier mathématiquement et expérimentalement le recours à l'early stopping ? (Q5)

Mathématiquement, l'early stopping correspond à un moment où l'erreur de généralisation (bias + variance) atteint son minimum dans le compromis biais-variance. Expérimentalement, nous observons que pour $h = 1024$, la précision test atteint son maximum à l'époque 10 (87.00%), justifiant l'arrêt à ce point.

8.1.6 En quoi l'implémentation concise masque-t-elle certains détails de bas niveau ? (Q6)

L'implémentation concise utilise des abstractions comme `nn.Sequential` et `nn.Linear` qui masquent plusieurs mécanismes fondamentaux :

— **La gestion des paramètres :** La création manuelle des matrices de poids W et des vecteurs de biais b est automatisée.

— **L'enregistrement (Registration) :** Le processus par lequel un tenseur est déclaré comme paramètre entraînable (`nn.Parameter`) est géré en interne par les couches.

— **Les opérations explicites :** Les multiplications matricielles (`matmul`) et l'application élément par élément de l'activation sont encapsulées.

— **L'initialisation des poids :** L'initialisation He est appliquée automatiquement, masquant l'importance du choix de σ.

8.1.7 Quels sont les avantages et les limites pédagogiques d'une implémentation from scratch ? (Q7)

L'approche "from scratch" est un outil pédagogique puissant mais limité pour la production :

— **Avantages :** Elle force à comprendre la structure mathématique réelle et le flux des tenseurs (shapes). Elle permet de visualiser l'impact direct de l'initialisation sur la stabilité des gradients. Elle explicite le rôle de chaque opération.

— **Limites :** Cette méthode devient rapidement « désordonnée » et difficile à étendre. Ajouter une couche impose un renommage fastidieux des paramètres. De plus, elle empêche le framework d'appliquer des optimisations de performance spécifiques.

8.2 Limites des MLP en vision par ordinateur

8.2.1 Perte de structure spatiale

Les MLP opèrent sur des images vectorisées, ignorant complètement la structure spatiale. Un pixel à une position donnée n'a pas de relation privilégiée avec ses voisins, alors que dans une image, les motifs locaux (bords, textures) sont essentiels pour la classification.

8.2.2 Quand un CNN serait plus pertinent

Les Convolutional Neural Networks (CNN) sont plus adaptés car :

— Ils exploitent la structure spatiale via des convolutions locales qui conservent la topologie.

— Ils sont naturellement invariants aux translations grâce au partage des poids.

— Le partage des poids réduit considérablement le nombre de paramètres.

Pour Fashion-MNIST, un CNN simple atteint 92-93% de précision, dépassant les 87.00% obtenus par notre MLP.

8.3 Améliorations méthodologiques possibles

Pour améliorer la généralisation, plusieurs techniques peuvent être envisagées :

- **Dropout** : Désactive aléatoirement 50% des neurones pendant l'entraînement.
- **Pénalisation L2 (weight decay)** : Ajoute une pénalité quadratique aux poids.
- **Optimiseurs adaptatifs** : Adam pourrait remplacer SGD pour une convergence plus rapide.
- **Augmentation de données** : Rotation, translation, zoom pour enrichir le jeu d'entraînement.
- **Batch Normalization** : Normalise les activations pour stabiliser l'entraînement.

9 Conclusion et perspectives

Ce mini-projet nous a permis d'explorer en profondeur les mécanismes des perceptrons multicouches, de l'implémentation bas niveau aux API haut niveau.

Principaux résultats :

- La meilleure configuration est `num_hiddens`=1024 et `lr`=0.1, atteignant une précision test de **87.00%**.
- Le sous-apprentissage est observé pour `num_hiddens` ≤ 64 (précision test $\leq 86.10\%$).
- Le sur-apprentissage apparaît pour `num_hiddens` ≥ 512 avec un gap de 2.62-2.78%.
- L'implémentation concise est plus lisible et maintenable, avec une meilleure précision maximale (86.34% contre 85.73%).

Perspectives :

- Ajouter une régularisation Dropout pour réduire le surapprentissage observé pour les grands modèles.
- Comparer avec un CNN pour quantifier l'avantage de la prise en compte de la structure spatiale.
- Visualiser les activations de la couche cachée via PCA/t-SNE.
- Explorer des architectures plus profondes (2-3 couches cachées).

10 Reproductibilité des résultats

L'ensemble des expériences présentées dans ce rapport a été réalisé dans un environnement contrôlé pour garantir la reproductibilité.

10.1 Seed aléatoire

Pour éliminer la variabilité due à l'initialisation aléatoire des poids, une seed fixe a été utilisée :

```
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)
```

10.2 Environnement technique

- **Matériel** : NVIDIA GeForce GTX 1650 (GPU)
- **Système** : Windows 11, Docker Desktop
- **Python** : 3.10
- **PyTorch** : 2.2.1+cu121
- **Bibliothèques** : torchvision 0.17.1, matplotlib 3.10.0, numpy 2.0.2

10.3 Instructions de reproduction

Pour reproduire exactement les résultats :

1. Installer Python 3.10 et les dépendances listées ci-dessus
2. Exécuter le notebook `MLP_Project.ipynb` dans l'ordre des cellules
3. La première cellule fixe la seed à 42
4. Les résultats obtenus doivent correspondre à ceux des tableaux 1 et 3

Toutes les expériences ont été exécutées une seule fois avec cette configuration.

Références

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[2] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2021). *Dive into Deep Learning*. Cambridge University Press.

[3] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST : a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv :1708.07747*.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers : Surpassing Human-Level Performance on ImageNet Classification. *ICCV*.

[5] Paszke, A., et al. (2019). PyTorch : An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*.

A Code source et reproductibilité

A.1 Lien vers le code source

L'ensemble du code est disponible sur GitHub : `https://github.com/Rayane-Loukhmi/MLP-FashionMNIST-Project`

Le notebook peut être exécuté directement dans Google Colab : `https://colab.research.google.com/github/Rayane-Loukhmi/MLP-FashionMNIST-Project/blob/main/MLP-DeepLearning-version_final.ipynb`

B Code complet de l'implémentation from scratch

```python
class MLPScratch(d2l_torch.Classifier):
    def __init__(self, num_inputs, num_outputs, num_hiddens, lr,
       sigma=0.01):
        super().__init__()
        self.save_hyperparameters()
        self.W1 = nn.Parameter(torch.randn(num_inputs, num_hiddens)
            * sigma)
        self.b1 = nn.Parameter(torch.zeros(num_hiddens))
        self.W2 = nn.Parameter(torch.randn(num_hiddens, num_outputs
           ) * sigma)
        self.b2 = nn.Parameter(torch.zeros(num_outputs))

    def forward(self, X):
        X = X.reshape((-1, self.num_inputs))
        H = torch.relu(torch.matmul(X, self.W1) + self.b1)
        return torch.matmul(H, self.W2) + self.b2
```

Listing 9 – MLPScratch complet

C Code de l'implémentation concise

```python
class MLPConcise(d2l_torch.Classifier):
    def __init__(self, num_outputs, num_hiddens, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.LazyLinear(num_hiddens),
            nn.ReLU(),
            nn.LazyLinear(num_outputs)
        )

    def forward(self, X):
        return self.net(X)
```

Listing 10 – MLPConcise complet

D Code de la campagne expérimentale

```python
def compare_hiddens(hidden_list, data, max_epochs=10, lr=0.1):
    results = {}
    for h in hidden_list:
        model = MLPConcise(num_outputs=10, num_hiddens=h, lr=lr)
        trainer = d2l_torch.Trainer(max_epochs=max_epochs)
        trainer.fit(model, data)
        train_acc = d2l_torch.accuracy(model, data.train_dataloader
           ())
```

```
        test_acc = d2l_torch.accuracy(model, data.test_dataloader())
        results[h] = {'train': train_acc, 'test': test_acc, 'gap': train_acc - test_acc}
        print(f"h={h:>4} | train={train_acc:.2%} | test={test_acc:.2%} | gap={train_acc-test_acc:.2%}")
    return results

def compare_lr(lr_list, data, max_epochs=10, num_hiddens=256):
    results = {}
    for lr in lr_list:
        model = MLPConcise(num_outputs=10, num_hiddens=num_hiddens, lr=lr)
        trainer = d2l_torch.Trainer(max_epochs=max_epochs)
        trainer.fit(model, data)
        train_acc = d2l_torch.accuracy(model, data.train_dataloader())
        test_acc = d2l_torch.accuracy(model, data.test_dataloader())
        results[lr] = {'train': train_acc, 'test': test_acc}
        print(f"lr={lr:>5} | train={train_acc:.2%} | test={test_acc:.2%}")
    return results
```

Listing 11 – Fonction de comparaison des hyperparamètres